This is the peer reviewed version of the following article: F. Coutinho, E. Ogasawara, D. de Oliveira, V. Braganholo, A. A. B. Lima, A. M. R. Dávila, M. Mattoso. Many task computing for orthologous genes identification in protozoan genomes using Hydra. Concurrency and Computation: Practice and Experience 23(17): 2326–2337 (2011), which has been published in final form at https://doi.org/10.1002/cpe.1786. This article may be used for non-commercial purposes in accordance with Wiley Terms and Conditions for Use of Self-Archived Versions.

Many Task Computing for Orthologous Genes Identification in Protozoan Genomes using Hydra*

Fábio Coutinho[1], Eduardo Ogasawara[1,2], Daniel de Oliveira[1], Vanessa Braganholo[3], Alexandre A. B. Lima[1], Alberto M. R. Dávila[4,5], Marta Mattoso[1]

[1]*COPPE/Federal University of Rio de Janeiro - Rio de Janeiro – Brazil*

[2]*Federal Center of Technological Education - Rio de Janeiro – Brazil*

[3]*Fluminense Federal University - Niterói – Brazil*

[4]*Oswaldo Cruz Institute – FIOCRUZ – Rio de Janeiro - Brazil*

[5]*Computational and Systems Biology Pole – FIOCRUZ – Rio de Janeiro - Brazil*

SUMMARY

One of the main advantages of using a scientific workflow management system (SWfMS) is to orchestrate data flows among scientific activities and register provenance of the whole workflow execution. Nevertheless, the execution control of distributed activities in high performance computing (HPC) environments by SWfMS presents challenges such as steering control and provenance gathering. Such challenges may become a complex task to be accomplished in bioinformatics experiments, particularly in Many Task Computing (MTC) scenarios. This paper presents a data parallelism solution for a bioinformatics experiment supported by Hydra, a middleware that bridges SWfMS and HPC to enable workflow parallelization with provenance gathering. Hydra MTC parallelization strategies can be registered and reused. Using Hydra, provenance may also be uniformly gathered. We have evaluated Hydra using an Orthologous Gene Identification workflow. Experimental results show that a systematic approach for distributing parallel activities is viable, sparing scientist time and diminishing operational errors, with the additional benefits of distributed provenance support.

1. INTRODUCTION

Large scale scientific experiments are comprised by a set of activities that consumes and produces data and is commonly represented as scientific workflows [1]. These workflows can be complex and their execution may last for hours, days or even months. Such executions aim to confirm or refute the hypothesis of a scientific experiment. In fact, scientists may need to repeatedly execute their workflows using different input parameter values or different input datasets to validate a hypothesis. These variations should be controlled with each workflow execution specification being associated with its corresponding execution data. When the workflow is defined using scripts or low level programming interfaces, scientists can be lost when several variations for the same experiment are being tested. According to Zhao, Raicu and Foster [2] there is "a need of common generic infrastructures and platforms in the science domain for workflow administration, scheduling, execution, monitoring,

* This work is partially sponsored by CNPq, CAPES and FAPERJ.

provenance tracking, etc". In other words, the scientific experiment life cycle must be coordinated as a whole in a structured way.

There exist many Scientific Workflow Management Systems (SWfMS), such as VisTrails, Pegasus and Swift [3-5]. They are used to orchestrate scientific data flows by controlling the execution of the workflow and gathering provenance data alongside it. SWfMS are increasingly providing more support in workflow design, reuse, version control and provenance gathering. Currently, those strong in workflow design and provenance gathering, such as VisTrails [3], lack HPC support. On the other hand, SWfMS specifically built for grids, such as Pegasus [4] and Swift [5], are strong in HPC support but do not provide version control to help workflow evolution design, nor the corresponding provenance support.

Provenance [6] is a key feature of SWfMS, since the analysis of scientific results can only be evaluated if provenance data has been collected in a structured manner during all experiment phases. According to Freire *et al.* [6] the provenance is as important as the results of a scientific experiment. There are two distinct forms of provenance: Prospective (related to the definition of the experiment) and Retrospective (related to the data products of an execution of an experiment) [6]. Retrospective provenance is managed by the SWfMS execution engine. When workflow activities are distributed to run in remote HPC, the scheduling of these activities are done by specific software rather than the workflow engine. SWfMS are not aware of the parallel strategy of the activity execution. Consequently, remote execution control and provenance registry of the parallel activities can be limited from the SWfMS point of view. Therefore, combining systems that are rich in workflow design and version control with systems that can distribute activities to run in remote HPC is an important step towards managing large scale experiments.

This paper explores the data parallelism of workflow activities in HPC environments using the Many Task Computing (MTC) paradigm [7] to distribute workflow activities in remote HPC environments. We have implemented data parallelism mechanisms in our Hydra middleware [8]. Hydra bridges the gap between SWfMS and the HPC environment. Hydra supports workflow parallelization and provenance gathering using MTC paradigm. In previous work [9], Hydra data parallelism components and architecture were initially evaluated using a small workflow for genome alignment of the *plasmodium falciparum* using SwissProt. Although that workflow has indicated that Hydra data parallelism could a promising solution, the experiment neither produced nor handled a large dataset and the sequence comparison using BLAST [10] was quite simple. Nevertheless, many existing applications must manipulate increasingly large volumes of data. This is referred to as data deluge [11]. We address this issue in this paper by measuring how Hydra scaled when managing a large volume of data using a new experiment based on the identification of orthologous groups in Protozoan genomes. This new experiment compares protein sequences from a set of target organisms. We have considered 22 different organisms. They were used to produce best reciprocal similarity pairs. In addition, this new experiment comprises two activities (BLAST and MCL), one running in a MTC environment (BLAST) and the other in a sequential SWfMS environment (MCL).

In this work, we have enriched Hydra by implementing new components for data parallelization and validated them in a real large scale experiment. Using these components, a parallelization strategy can be registered and reused, and provenance may be gathered in the HPC environment and integrated with the provenance captured by the SWfMS. The goal is to reduce the complexity involved in designing and managing activity/workflow parallel executions within scientific experiments. The main contributions of this paper are: (i) providing data parallelism in Hydra by building cartridges for data fragmentation; (ii) experimenting with Hydra in the orthologous identification domain that requires data parallelization and produces a large volume of data. By using the Orthologous Genes Identification workflow, we show how to improve workflow design and how to gather provenance when distributing workflow activities to remote HPC environments. We show that it is simple for scientists to execute experiments that "transit"

from a sequential environment to a MTC one without having a background in parallel programming techniques such as MPI or OpenMP.

The remainder of the paper is organized as follows. Section 2 presents the Hydra data parallelism strategy. Section 3 presents the case study in the Orthologous Genes Identification used to evaluate Hydra. Section 4 shows Hydra experimental results and the provenance data extracted during workflow execution. Section 5 discusses related work. Section 6 concludes.

2. DATA PARALLELISM WITH HYDRA

Hydra [12,9] is a middleware capable of distributing, controlling and monitoring the execution of activities of a scientific workflow in a MTC environment. Hydra provides support for different types of activity parallelization with little effort and minimal changes to an original sequential scientific workflow. Since Hydra architectural characteristics were previously detailed [12,9], this section focuses on the data parallelism strategy. To better understand it, we first formally define a scientific workflow.

Our formalization was adapted from Meyer *et al.* [13]. A workflow *Wf* is represented by a quadruple (*A*,*Pt*,*I*,*O*) where: *A* is a set $\{a_1, a_2, \ldots, a_n\}$ of activities that constitute the workflow *Wf*; *I* is a set $\{i_1, i_2, \ldots, i_m\}$ of input data; *Pt* is a set $\{pt_1, pt_2, \ldots, pt_r\}$ of *Wf* parameters and *O* is a set of sets $\{O_1, O_2, \ldots, O_n\}$, where O_j is a set of output data for activity a_j. Each parameter in *Pt*, as well as each input data in *I*, may assume values in the domain associated with it. D_{pti} denotes the domain of possible values for parameter pt_i, and D_{ij} denotes the domain of possible values for input data i_j. There are several possibilities for parallelizing workflows [13,14]. Although Hydra also provides parameter sweep [14] capabilities, in this paper we focus on data parallelism [13]. These two types of parallelism are very common and may represent a barrier for the scientist to control, gather and register workflow provenance.

Parameter sweep parallelism is characterized by many simultaneous executions of one activity of the workflow *Wf*, each execution using different values for the parameters *Pt*. For example, suppose that a given workflow *Wf* has parameters pt_1 and pt_2. Suppose also that the domain of possible values for parameters pt_1 and pt_2 is D_{pt1} = {x, x'} and D_{pt2} = {y, y'}. One instance of one activity a_i of *Wf* may consume values *x* and *y* for parameters pt_1 and pt_2 respectively, and another instance may consume values *x'* and *y'*. This can be achieved by allocating a different configuration of values for the parameters in *Pt* at each node of the MTC environment as detailed in a previous work [12].

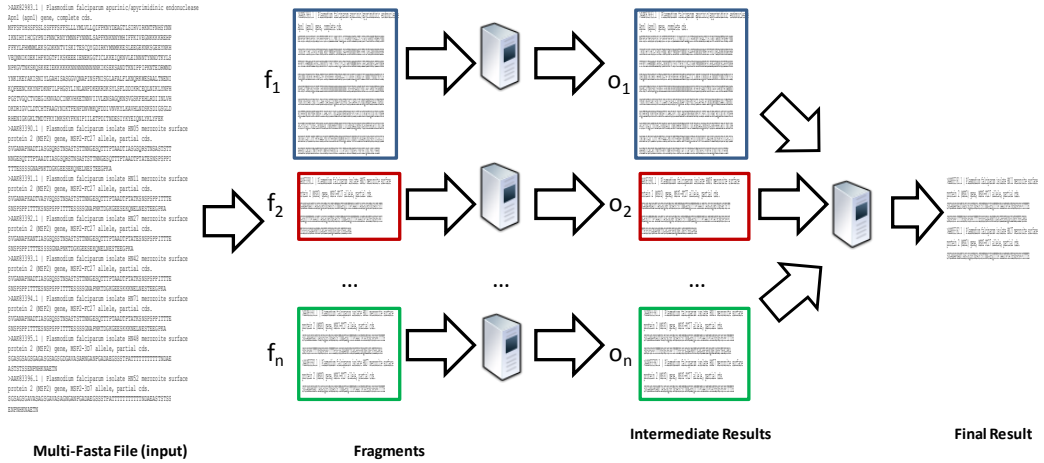


Figure 1 – Data Parallelism in a Bioinformatics Experiment

Data parallelism can be characterized by the simultaneous execution of many instances of one activity a_i of the workflow *Wf*, each instance processing a different subset of the entire input data. This parallelism can be achieved by replicating the activity a_i in all nodes of the MTC environment and giving each node a specific subset of the entire input data. As an example, suppose a workflow *Wf* with

input data I. Also, assume that the domain of I is $D_I = \{1,3,5,7,9,12\}$. We could define fragments of D_I by using a fragmentation function *f*. In the example, we could define that the fragments of D_I resulting from $f(D_I)$ are $D_{I\text{-}f1} = \{1,3\}$, $D_{I\text{-}f2} = \{5,7\}$, and $D_{I\text{-}f3} = \{9,12\}$. Notice that the result of the fragmentation function needs to be disjoint. Figure 1 illustrates this kind of parallelism using a real fragmentation of a bioinformatics experiment presented in a previous work [9]. In Figure 1 a multi-Fasta file used as input is fragmented in several parts (each one containing one or more DNA sequences) and the same activity is replicated in *n* nodes. Each replicated activity consumes a particular fragment and produces an intermediate output which is further aggregated in order to produce the final result.

3. CASE STUDY: WORKFLOW FOR ORTHOLOGOUS GENES IDENTIFICATION IN PROTOZOAN GENOMES

This section describes the bioinformatics experiment used to evaluate Hydra. This case study focuses on the identification of orthologous genes in protozoan genomes. It aims to analyze all protein sequences in Protozoan genomes to identify the ones presenting similar function between several species and probably a common ancestor.

3.1 OrthoMCL Conceptual Workflow

In recent years, mainly because of the success of large scale DNA sequencing, a great deal of time and effort has been spent by scientists in developing evolutionary analyses of organisms. Evolutionary analyses is a field of biology that focuses on analyzing the origin, change, multiplication and diversity over time of species, based upon similarities and differences in their physical (phenotype) and/or genetic (genotype) characteristics. One of the most important subjects in evolutionary analysis is the identification of homologous genes, especially orthologs. Indeed, this research issue has occupied a remarkable position in computational genomics and several methods for orthologous identification (*e.g.* InParanoid [15], Z1 Hundred [16], OrthologID [17], OrthoSearch [18] and OrthoMCL [19]) have been proposed. OrthoMCL provides a scalable method for identifying orthologous groups across multiple eukaryotic taxa, using a Markov Cluster algorithm [20] to cluster (putative) orthologs and paralogs. The identification of orthologous groups is very useful for studies on gene and genome evolution, which is the focus of this case study. The OrthoMCL workflow comprises BLAST [10] and MCL [20] activities and it is conceptually illustrated by Figure 2. This workflow takes as input a set of Fasta files containing protein sequences from distinct target organisms. These files are merged, generating one single multi-Fasta file. Next, this multi-Fasta file is submitted to BLAST to be checked against itself. The BLAST result is parsed to find reciprocal best similarity pairs (between and within species). After this phase, a graph, whose nodes represent protein sequences, is generated and the weighted edges represent orthologous and paralogous relationships. The graph is represented by a symmetric similarity matrix to which the MCL algorithm is applied. Finally, MCL provides a method for separating diverged paralogs and distant orthologs mistakenly assigned based on weak reciprocal best hits.



Figure 2 - The OrthoMCL Workflow

3.2 Experiment Setup

In the experiment presented in this paper we have considered the following proteomes to be analyzed by OrthoMCL workflow using Hydra middleware: *Babesia bovis*, *Cryptosporidium parvum*, *Cryptosporidium hominis*, *Cryptosporidium muris*, *Entamoeba dispar*, *Entamoeba histolytica*, *Giardia lamblia*, *Leishmania braziliensis*, *Leishmania infantum*, *Leishmania major*, *Plasmodium falciparum*,

Plasmodium knowlesi, *Plasmodium berghei*, *Plasmodium chabaudi*, *Plasmodium yoelii*, *Plasmodium vivax*, *Theileria annulata*, *Trypanosoma brucei*, *Trypanosoma cruzi*, *Theileria parva*, *Toxoplasma gondii*, and *Trichomonas vaginalis.*

The genomes of these 22 aforementioned organisms are being analyzed as part of ProtozoaDB [21] by the BiowebDB consortium (http://www.biowebdb.org) at Oswaldo Cruz Institute (FIOCRUZ). The evolutionary analysis of these organisms using OrthoMCL demands a complex process that requires high performance environments and parallel techniques to produce a result in a feasible time. In addition, it is a serious candidate to benefit from MTC paradigm provided by Hydra middleware. In the context of the OrthoMCL workflow, we considered using the data parallelism mechanisms, provided by Hydra, to execute BLAST. The execution time of BLAST represents the major bottleneck in the workflow. So, the proposal is to run BLAST in parallel using Hydra, with the rest of the OrthoMCL workflow executing sequentially outside Hydra (in the scientist's workstation).

3.3 Data parallelism applied to BLAST using Hydra

Figure 3 presents a comparative view between sequential (top) and parallel (bottom) OrthoMCL workflow execution. Our strategy for parallelizing BLAST activity using Hydra consists of breaking the OrthoMCL workflow into two parts: *OrthoMCL_preBLAST* and *OrthoMCL_postBLAST*. The former includes tasks executed before starting BLAST execution, such as aggregating distinct Fasta files, constructing formatted database and generating index and temporary files. The latter comprises tasks executed immediately after BLAST execution, such as parsing BLAST result, selecting best similarity pairs and the MCL execution. In our solution, the (sequential) OrthoMCL activity was rewritten as two new scripts that are responsible for activities launched at the pre and post BLAST stage. In section 3.1 we discuss in detail the issues related to BLAST parallel execution using Hydra middleware.

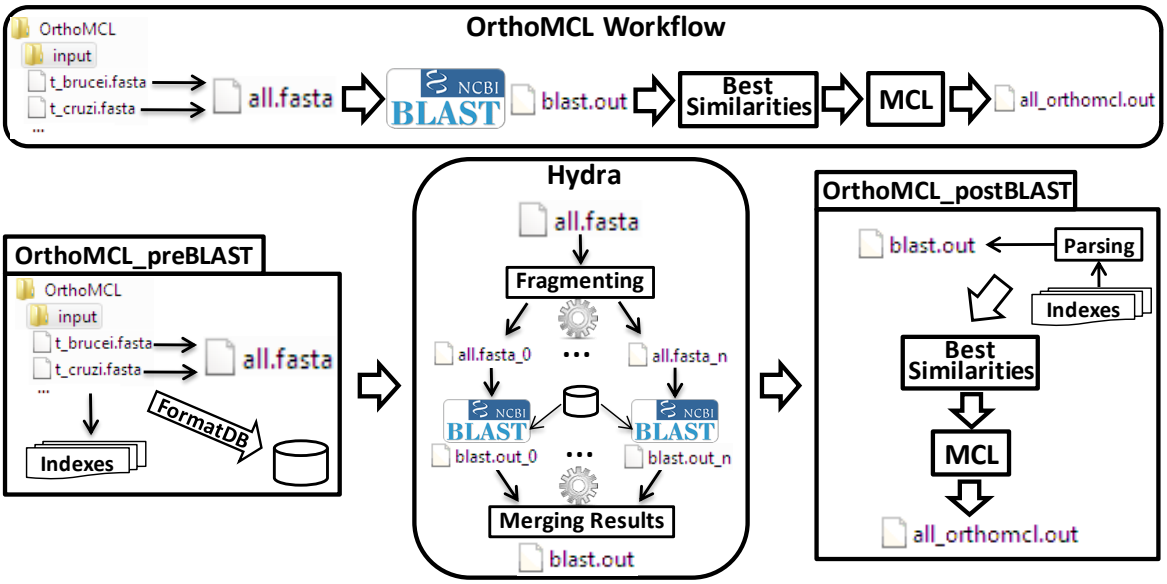


Figure 3 – Sequential (top) and parallel (bottom) view of OrthoMCL workflow

Data parallelism using BLAST involves the discussion of important issues, for example, whether to fragment the input query file and/or the database(s). There are at least three possible fragmentation scenarios to be considered for BLAST parallel execution. In the first one, only the input query file is fragmented. This way, each execution node takes a replica of the BLAST database and consumes a fragment containing a subset of the input sequences. Finally, the results are combined into a single output file. In the second scenario, only the database is fragmented and the entire input file is replicated in each execution node. Then, each node compares the entire input file against a portion of the database.

This scenario can generate some distortions related to statistical information. In order to correct this drawback, some parameters should be taken into account on BLAST invocation. One advantage of reducing database size is to execute less disk operations. In the third scenario, scientists may combine input file and database fragmentation, considering all of the aforementioned issues.

In our experiment, we chose to fragment only the input file (containing the sequences from all organisms) in order to use it in agreement with the data parallelism strategy proposed by Hydra. This solution is implemented by a specific cartridge that we have developed to fragment Fasta files that represent either DNA or protein sequences, in which nucleotides or amino acids are represented using single-letter codes. Each fragment is generated following a specific fragmentation rule. The Fasta fragmentation cartridge is based on the program FastaSplitter [9].

The fragmentation algorithm consists of two main phases: (a) naïve fragmentation; and (b) adjustment of the data fragments. The Naïve fragmentation phase is responsible for fragmenting the input Fasta file into parts (f_1, f_2, f_3, ..., f_{n-1}, f_n) with the same size (in bytes). The number of fragments is defined by the scientist during the desktop SWfMS invocation. Consider, as an example, a Fasta file named *all.fa* (generated by OrthoMCL) with approximately 158 MB to be fragmented into 256 fragments. The fragmentation cartridge fragments the Fasta file in 256 fragments (f_1, f_2, f_3, ..., f_{255}, f_{256}) with approximately 0.61MB each. However, these fragments may not have any biological meaning, since a specific DNA sequence may be split into more than one fragment, resulting in incomplete or truncated sequences, which is not acceptable.

To avoid this inconsistency, the second phase of the algorithm adjusts the fragments in order to ensure they contain complete DNA sequences. To do so, it rearranges data in the fragments, moving sequences from one fragment to another. To clarify, consider a specific DNA Sequence #1 initially split in two different fragments named f_1 and f_2. During the adjustment phase, part of the DNA sequence #1 that was in fragment f_2 is moved to fragment f_1 that now contains the complete DNA sequence #1. To identify the parts that should be rearranged, the algorithm considers that the first line of each new sequence starts with a ">" symbol that is a convention of Fasta format. After BLAST execution on each fragment, Data Aggregation is performed. This component is implemented by a cartridge that merges BLAST results generated on each fragment at one single output to be used by OrthoMCL_postBLAST activity as presented in Figure 3.

4. EXPERIMENTAL RESULTS

This section presents the analysis of the results produced by the execution of the bioinformatics experiment used to evaluate Hydra middleware. The chosen experiment is the Orthologous Genes Identification presented in Section 3. The OrthoMCL workflow was invoked using SWfMS VisTrails [3] from scientist's desktop and Hydra middleware for data parallelism of BLAST activity using MTC paradigm in a cluster. The experiment was conducted using a 32-node SGI Altix ICE 8200 cluster, each node with dual Quad-Core Intel Xeon processors and 8 GB RAM. The cluster uses a shared disk (SGI Infinite Storage NAS 32TB) file system for data storage. The hardware is installed at the High Performance Computing Center, COPPE/UFRJ (www.nacad.ufrj.br). Hydra MTC infrastructure was configured to use Torque as the job scheduler [22] to setup the job submission of the serial version of BLAST. Hydra middleware provides a systematic approach for supporting data parallelism in scientific workflows, which is achieved through small configuration steps and workflow adjustments.

4.1 Hydra Configuration

Before starting the execution of the experiment, some configurations are necessary. The *hydra.xml* file defines the parallel configuration for the workflow as presented in Figure 4.

```
<?xml version="1.0" encoding="UTF-8"?>
<Hydra>
  <database name="hydra" server="localhost" port="5432" username="FC54BCID" password="#98HGT"/>
  <HydraWorkflow name="Ortho-MCL" tag="OrthoMCL.vt" expdir="/home/users/ogasawara/orthomcl">
    <HydraActivity tag="All-against-all-BlastP" name="BLAST" type="MAP">
                    <TemplateDir><InstrumentedFile fileName="Blast.cmd" tag="Blast"/></TemplateDir>
                    <HydraActivityExecution>
                            <ExperimentDir value="/scratch/ogasawara/exp64"/>
                            <Fragmentation cartridge="hydra.Blast.DataCartridge" input="all.fa" output="all.out"/>
                            <SimpleExecution><parameter name="FASTAFILE" type="File"/></SimpleExecution>
                            <WorkflowFile value="OrthoMCL.vt"/>
                    </HydraActivityExecution>
    </HydraActivity>
  </HydraWorkflow>
</Hydra>
```

Figure 4 – Setup of Blast Activity in Hydra

This XML file is used to setup Hydra. It is transferred from the scientist's desktop to the MTC environment. The information related to the fragments produced by Hydra represents part of the prospective provenance [6] (which is detailed in the next sub-section). It contains important information such as parallelization model (data fragmentation), fragmentation cartridge, among others. The configuration files or command line parameters for each program were then instrumented using tags that are easy to understand and to implement. In fact, the Blast activity has an instrumented file that contains tags in the command line to invoke the Blast program as presented in Figure 5. These tags are similar to Java Server Pages (JSP) tags [23], which are used to differentiate each parameter or data fragment involved in the parallel execution. Hydra processes each instrumented file in a separated directory to support parallel execution. Each directory is related to a particular set of parameters.

```
blastall -p blastp -d /users/ogasawara/orthomcl/input/all.fa -i <%=DIREXP%>/all.fa_<%=IDX%> -o
<%=DIREXP%>/result_<%=IDX%>.txt -e 1e-5 -m 0 -a 1 -v 1000 -b 1000
```

Figure 5 – Instrumented file in Hydra for invocating BLAST

The scientist (or any other person involved with the experiment) should write these instrumented files to execute them in Hydra. However, this instrumented file can be easily written by non-specialists on grid or parallel computing, since it contains only the invocation of a sequential program with the reserved words and parameters already used by scientists. The only difference from the instrumented file to a regular invocation script is that the directories that contain the input data to be consumed and file names have to be tagged in order to be resolved by Hydra, which substitutes all tags for actual values. For example, consider the instrumented file presented in Figure 5. Suppose that a scientist is parallelizing BLAST (and two fragments extracted from *all.fasta* file). Hydra replaces the tags <%=DIREXP%> and <%=IDX%> during execution, changing <%=DIREXP%> with the name of the directory and <%=IDX%> with the identifier of the generated fragment. This replacement is automatically executed, with no need for scientists' interference, *i.e.* this task is transparent for scientists and the distribution of activities does not require any expertise in grid or distributed computing from them. Although the instrumentation can be simple, this is problem-dependent. In fact, the instrumentation of the workflow can vary for different applications.

Once instrumented files are created and Hydra XML is set up, it is only necessary to make small changes to the sequential scientific workflow to support data parallelism. The main idea is to replace the activity to be parallelized in the sequential workflow by three Hydra activities (*Stage-in, Dispatcher, and Stage-out*). The *Stage-in* (previously known as Uploader [9]) activity uploads the multi-Fasta file to a shared disk in the cluster. When the data transfer is finished, the *Dispatcher* activity launches the

execution of the Blast activity on the cluster. Once the distributed execution is finished, the Dispatcher returns control to the SWfMS (in this case, VisTrails). The *Stage-out* component downloads the final results achieved with the distributed execution from the shared disk to the scientist's desktop.

Note that the original activities' input and output types are preserved when the new activities are placed in the workflow, *i.e.* the input type of the Blast [10] activity of Figure 6 is now the input type of the Stage-in activity in Figure 7. Similarly, the output type of the Blast (Figure 6) activity corresponds to the output type of the Stage-out activity in Figure 7. However, some adaptations on the code of the activity in the SWfMS (in this case, using Python) are needed. The advantage here is that scientists usually model and code their own workflows. Thus changing the code of the activity in the SWfMS is trivial for them, which may not be the case for parallel executions and mechanisms.

In Figure 7, the *Blast Dispatcher* invokes the distributed *Blast* activity in the cluster. All other activities (*Local Directory, Stage-in, Reciprocal Best Similarity Pairs, MCL and Stage-out*) are still executed in the scientists' desktop.

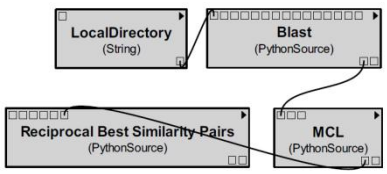


Figure 6 Sequential workflow

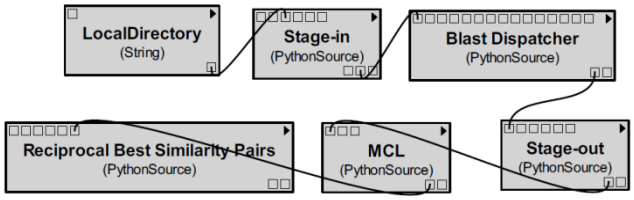


Figure 7 Parallel workflow with Hydra components

4.2 Hydra Execution

During workflow execution, the *Stage-in* activity is executed on VisTrails to transfer the multi-Fasta file required by BLAST activity. This file is transferred to the MTC environment (cluster). The *dispatcher* activity is then executed on VisTrails to invoke parallel execution of BLAST activities in a cluster using Hydra MTC components[†]. The Hydra MTC components automatically handle and create isolated directories that include all files for each distributed activity executed at the MTC environment. In the data fragmentation scenario, an individual directory is created for each data fragment to be processed. Hydra provenance mechanism registers each directory and its association to each specific fragment. Each individual activity execution is monitored. Hydra registers both retrospective (workflow execution times, data products produced and so on) and prospective (definition of workflow and activities that are captured from the *hydra.xml* file) provenance. Provenance is stored in a relational database managed by PostgreSQL. This database can be accessed from the nodes in the MTC environment without interfering in workflow execution time. After all activities are completed, the execution control returns to VisTrails. The *Stage-out* activity brings the results from the MTC environment back to VisTrails.

In the following, we present information about Hydra MTC execution obtained from the Hydra provenance database during the experiment as we increase the number of cores used in the cluster. The multi-Fasta file was approximately 158 MB and it was fragmented into 256 independent files of 0.61MB each. The *Stage-in* component took approximately 0.9 min (transfer time) to transfer the input file (158 MB) to the shared-disk file system. This is the first difference between this experiment and the previous one described by Coutinho *et al.* [9]. The input data size from the previous experiment [9] was 55% larger than the present one. After the execution of the distributed activities (each one consuming a specific fragment), the results (*.out* files) are stored in different directories (in the same shared-disk file

[†] Hydra components are detailed by Ogasawara *et al.* [12]

system) and then transferred from the MTC environment (cluster) to the SWfMS. During the distributed execution, the parallel tasks produced approximately 110 GB of output data altogether, which is considerably larger than the 30 MB produced in our previous experiment [9]. This difference allowed us to evaluate Hydra's capacity for handling a large dataset. To transfer these files back to scientist's desktop, the *Stage-out* component downloads the results. The output data transfer time was about 300 minutes in a local network (100 Mbps). These transfers presented a fixed time (0.9 min + 300 min) and they represent all the data transferred between the SWfMS (client) and the HPC environment (server).

The MTC execution time represents the total time needed to execute all parallel activities. With 64 cores, Hydra execution led to a speedup of 49.08 and with 128 cores, a speedup of 99.51[‡] was achieved.

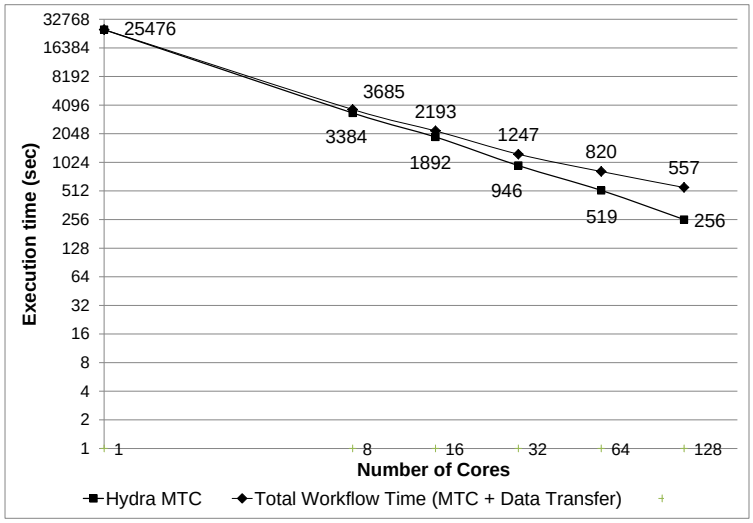


Figure 8 Workflow execution time

The execution time for Hydra MTC and the whole process of distributing the workflow (*Stage-in*, *Dispatcher*, and *Stage-out*) is represented by the "Total Workflow" line of Figure 8, where we can observe the overhead of transferring data. This way, the line with diamonds represents the total execution time of the distributed workflow, thus considering the effective execution of the distributed activities and the stage-in and stage-out time to and from the MTC environment. On the other hand, the "Hydra MTC" line represents only the execution time of the distributed activities (*Dispatcher*) without taking into account the transfer times (stage-in and stage-out). As mentioned before, stage-in and stage-out have a fixed time, since the input and output data size is fixed. As expected, we may observe in Figure 8 that as the number of cores increases the total execution time of distributed activities decreases. However, the transfer time remains the same, thus increasing the impact of transfer time on total execution time, as the number of cores increases.

The benefits of data parallelism are clear. However, depending on the multi-Fasta file taken as input, the stage-in and stage-out times may be huge. Thus, acquiring more nodes for execution does not bring benefits in the same proportion, particularly if the number of distributed activities is closer to the number of nodes. Often, the multi-Fasta file could have already been transferred or generated by a previous activity on the MTC environment. In such cases, the stage-in activity may be suppressed. Since Hydra uses a shared disk approach, data staging is only executed twice: from scientist's desktop to the cluster and *vice-versa*. However, to reduce the impact of data staging, it is preferable to transfer data files before starting the workflow execution. In addition, several input files can be consumed by two or more experiments, thus reducing the impact of data staging in future executions. As for the stage-out, some applications transform the produced data into another format to be analyzed through the MTC. In a

[‡] The speed-up is based on comparisons with executions using 1 core in the cluster

previous work [9] we have improved stage-in and stage-out mechanisms to reduce the impact of transferring data: each input and output file is compressed before being transferred over the network. Nevertheless we are still investigating better ways to improve data transfer time.

The information registered in the provenance database during the experiment is important to evaluate and reproduce the experiment execution. Important provenance data could be lost if a systematic controlled approach such as Hydra was not used. Scientists can access the provenance database to check which fragments were produced or submit specific queries such as "What are the directories of activities that produced a *.out* file greater than 450 MB as output?". This query can be translated to the following SQL statement in our provenance schema:

```
SELECT t.WorkspaceDir, o.OutputName FROM Task t, OutputValue ov, Output o WHERE
o.OutputID = ov.OutputID AND ov.TaskID = t.TaskID AND o.OutputName like '%.out'
AND ov.size > 450*1024;
```

Our provenance database is independent to any existing provenance mechanism of SWfMS. In this paper we integrated the distributed provenance captured by Hydra with local provenance registered by VisTrails. The ID of activity execution in VisTrails is sent to Hydra middleware, which associates provenance data captured in the cluster (directories, data products, parameters) with the dispatcher activity of VisTrails. This way, scientists are able to execute queries that are based in both local and distributed provenance.

5. RELATED WORK

The most similar approach to data parallelism in Hydra is MapReduce. MapReduce [24] is a model for executing a large range of applications in parallel in large clusters or in the cloud. MapReduce allows the programmer to define the minimal computational grain and the framework hides parallelism details. To use MapReduce, two main functions have to be programmed: (i) a map function that can be used to fragment input data and (ii) a reduce function that is used to merge or aggregate intermediary results to create a final result. Both map and reduce functions are problem-dependent. MapReduce is highly used in business applications such as document crawlers. Many scientific applications are also candidates to benefit from it. However, the MapReduce concept is disconnected from the concept of SWfMS, scientific experiments and provenance. In CloudBLAST [25], the MapReduce paradigm is used to parallelize bioinformatics tools and manage their execution. However, its approach is disconnected from the concept of scientific workflows and it is not concerned about capturing and analyzing provenance data.

Another similar approach is Swift/Falkon [26]. Falkon provides efficient dispatch and execution of many small tasks, according to the MTC paradigm in grids and large computer clusters. The main goal of Falkon is to solve problems like long queue times in batch-scheduled systems, slow job dispatch rates and poor scalability of shared file systems. Falkon has been used with Swift to provide MTC in scientific workflows. Hydra is also focused on providing MTC to workflows but aims to be independent from SWfMS. Hydra helps the scientist to achieve implicit parallelism to be further scheduled as MTC. In this way, Falkon can be seen as complementary to Hydra since it can play the role of the scheduler of Hydra. When comparing Swift/Falkon with Hydra, the focus of provenance gathering in Hydra is at the experiment rather than on isolated workflow instances, thus it includes provenance from abstract workflow specification and execution including MTC. Therefore, Swift could also benefit from Hydra.

Similarly to Hydra, Nimrod/K [27] is a set of add-on components for the Kepler SWfMS. Nimrod/K includes the new Kepler 'director' to orchestrate the execution of Kepler SWfMS for parameter sweeper on clusters, grids and clouds. Nimrod/K also includes an optimization component used to optimize the

distributed SWfMS. Nevertheless, Nimrod only works with Kepler and is limited to handling parameter sweep, thus data parallelism is not a supported operation.

Gunarathne *et al.* [8] performs a study of several cloud computing approaches by executing different biological experiments. In one of them the authors developed distributed pair-wise sequence alignment applications using the MapReduce programming model. The intersection between our work and [8], is that, on their paper, the authors executed an experiment for assembling genome fragments, similarly to the one executed in this paper. However, Gunarathne *et al.* were interested in analyzing the behavior of biological experiments in clouds, thus not focusing on provenance capturing and integration with existing SWfMS.

6. CONCLUSION

Data parallelism is an important technique used to increase performance in data intensive scientific experiments. This is the case for most bioinformatics workflows. However, scientists more concerned with biological results than with parallel programming techniques must expend a large amount of effort to exploit data parallelism. Hydra is a middleware with a set of components to be placed on the scientific workflow specification modeled in any SWfMS (VisTrails, for example) to promote parallelization of activities following MTC paradigm in a uniform and controlled way. By using Hydra's set of components it is possible to achieve data parallelism.

Data fragmentation components are usually domain-specific. In other words, a new fragmentation rule is needed if the problem being solved changes. A fragmentation rule for a Fasta file is completely different from a fragmentation rule for another file format. We have implemented cartridges, which are extensible components to perform fragmentation rules. It means that Hydra provides ways for the scientist to choose a suitable fragmenter for the input data. Although a cartridge is also domain-specific, such as the MapReduce model, its concept is flexible, since it can be dynamically changed and it just involves the setup of a program that is capable of fragmenting a data file into a number of parts. Once the input data is fragmented, Hydra is responsible for automatically associating activities to each data fragment in the MTC environment. Hydra dispatches and monitors the execution of each one of these activities. Once all generated activities are finished, Hydra collects the distributed provenance to promote integration with the provenance schema of the SWfMS. Transferring data between the SWfMS and MTC environment, however, may be time consuming. To cope with this task, Hydra stage-in and stage-out components compress data prior to transfer and uncompress them after their transfer, thus reducing the transfer time [9].

In this paper we explored data parallelism in Hydra by modeling and executing in parallel a bioinformatics workflow for the identification of orthologous genes in protozoan genomes. This workflow aims to analyze protein sequences to identify regions that have similar function between several species and probably a common ancestor. The workflow rearranges the sequences of DNA to identify regions of similarity that are a consequence of evolutionary relationships between the sequences. We have considered 22 different organisms in our analysis. The workflow consists of two main activities: BLAST and MCL. The BLAST activity was parallelized while the MCL was executed sequentially. By executing this new experiment we aimed at measuring how Hydra scaled when dealing with a larger volume of output data compared to our previous work [9]. We have setup a fragmentation cartridge based on FASTA format using the FASTASplitter program that encapsulates the fragmentation rules. Our experiment with Hydra data fragmentation has been implemented on top of the VisTrails SWfMS, although Hydra components may be deployed in any SWfMS. Hydra MTC layer is deployed in SGI Altix cluster of the High Performance Computer Center of Federal University of Rio de Janeiro. By analyzing the performance of this architecture using larger datasets regarding data parallelism, we have reinforced the benefits (performance and usability associated with integrated provenance gathering) of

using Hydra middleware. As future work, we intend to evaluate Hydra performance against other parallel approaches such as MapReduce implementations (*e.g.* Hadoop) and Falkon.

7. REFERENCES

[1] M. Mattoso, C. Werner, G.H. Travassos, V. Braganholo, L. Murta, E. Ogasawara, D. Oliveira, S.M.S.D. Cruz, and W. Martinho, 2010, Towards Supporting the Life Cycle of Large Scale Scientific Experiments, *IJBPIM*, v. 5, n. 1, p. 79–92.

[2] Y. Zhao, I. Raicu, and I. Foster, 2008, Scientific Workflow Systems for 21st Century, New Bottle or New Wine?, In: *IEEE Congress on Services*, p. 467-471

[3] S.P. Callahan, J. Freire, E. Santos, C.E. Scheidegger, C.T. Silva, and H.T. Vo, 2006, VisTrails: visualization meets data management, In: *Proc. SIGMOD*, p. 745-747, Chicago, Illinois, USA.

[4] E. Deelman, G. Mehta, G. Singh, M. Su, and K. Vahi, 2007, "Pegasus: Mapping Large-Scale Workflows to Distributed Resources", *Workflows for e-Science*, Springer, p. 376-394.

[5] Y. Zhao, M. Hategan, B. Clifford, I. Foster, G. von Laszewski, V. Nefedova, I. Raicu, T. Stef-Praun, and M. Wilde, 2007, Swift: Fast, Reliable, Loosely Coupled Parallel Computation, In: *IEEE Services*, p. 206, 199, Salt Lake City, USA.

[6] J. Freire, D. Koop, E. Santos, and C.T. Silva, 2008, Provenance for Computational Tasks: A Survey, *Computing in Science and Engineering*, *v.10*, n. 3, p. 11-21.

[7] I. Raicu, I. Foster, and Yong Zhao, 2008, Many-task computing for grids and supercomputers, In: *Workshop on Many-Task Computing on Grids and Supercomputers*, p. 1-11, Austin, Texas.

[8] T. Gunarathne, T. Wu, J. Qiu, and G. Fox, 2010, Cloud computing paradigms for pleasingly parallel biomedical applications, In: *Proc. of the 19th HPDC*, p. 460–469, New York, NY, USA.

[9] F. Coutinho, E. Ogasawara, D. Oliveira, V. Braganholo, A.A.B. Lima, A.M.R. Dávila, and M. Mattoso, 2010, Data Parallelism in Bioinformatics Workflows Using Hydra, In: *ECMLS 2010*, Chicago, Illinois, United States.

[10] S.F. Altschul, W. Gish, W. Miller, E.W. Myers, and D.J. Lipman, 1990, Basic Local Alignment Search Tool, *Journal of Molecular Biology*, v. 215, n. 3, p. 403-410.

[11] I. Gorton, P. Greenfield, A. Szalay, and R. Williams, 2008, Data-Intensive Computing in the 21st Century, *Computer*, v. 41, n. 4, p. 30-32.

[12] E. Ogasawara, D. Oliveira, F. Chirigati, C.E. Barbosa, R. Elias, V. Braganholo, A. Coutinho, and M. Mattoso, 2009, Exploring many task computing in scientific workflows, In: *MTAGS 09*, p. 1-10, Portland, Oregon.

[13] L. Meyer, D. Scheftner, J. Vöckler, M. Mattoso, M. Wilde, and I. Foster, 2007, "An Opportunistic Algorithm for Scheduling Workflows on Grids", *VECPAR 2006*, 1 ed, p. 1-12.

[14] E. Walker and C. Guiang, 2007, Challenges in executing large parameter sweep studies across widely distributed computing environments, In: *Workshop on Challenges of large applications in distributed environments*, p. 11-18, Monterey, California, USA.

[15] G. Ostlund, T. Schmitt, K. Forslund, T. Köstler, D.N. Messina, S. Roopra, O. Frings, and E.L.L. Sonnhammer, 2010, InParanoid 7: new algorithms and tools for eukaryotic orthology analysis, *Nucleic Acids Research*, v. 38, n. Database issue (Jan.), p. D196-203.

[16] J.P. Comet, J.C. Aude, E. Glémet, J.L. Risler, A. Hénaut, P.P. Slonimski, and J.J. Codani, 1999, Significance of Z-value statistics of Smith-Waterman scores for protein alignments, *Computers & Chemistry*, v. 23, n. 3-4 (Jun.), p. 317-331.

[17] J.C. Chiu, E.K. Lee, M.G. Egan, I.N. Sarkar, G.M. Coruzzi, and R. DeSalle, 2006, OrthologID: automation of genome-scale ortholog identification within a parsimony framework, *Bioinformatics*, v. 22, n. 6 (Mar.), p. 699-707.

[18] S.M.S.D. Cruz, V. Batista, A.M.R. Dávila, E. Silva, F. Tosta, C. Vilela, M.L.M. Campos, R. Cuadrat, D. Tschoeke, et al., 2008, OrthoSearch: a scientific workflow approach to detect distant homologies on protozoans, In: *Proc. of the ACM SAC*, p. 1282-1286, Fortaleza, Ceara, Brazil.

[19] L. Li, C.J. Stoeckert, and D.S. Roos, 2003, OrthoMCL: Identification of Ortholog Groups for Eukaryotic Genomes, *Genome Research*, v. 13, n. 9, p. 2178-2189.

[20] A.J. Enright, S.V. Dongen, and C.A. Ouzounis, 2002, An efficient algorithm for large-scale detection of protein families, *Nucleic Acids Research*, v. 30, n. 7, p. 1575-1584.

[21] A.M.R. Dávila, P.N. Mendes, G. Wagner, D.A. Tschoeke, R.R.C. Cuadrat, F. Liberman, L. Matos, T. Satake, K.A.C.S. Ocaña, et al., 2008, ProtozoaDB: dynamic visualization and exploration of protozoan genomes, *Nucleic Acids Research*, v. 36, n. Database issue (Jan.), p. D547-D552.

[22] A. Bayucan, R.L. Henderson, and J.P. Jones, 2000, Portable Batch System Administration Guide

[23] H. Bergsten, 2003, *JavaServer pages*. O'Reilly Media, Inc.

[24] J. Dean and S. Ghemawat, 2008, MapReduce: simplified data processing on large clusters, *Commun. ACM*, v. 51, n. 1, p. 107-113.

[25] A. Matsunaga, M. Tsugawa, and J. Fortes, 2008, CloudBLAST: Combining MapReduce and Virtualization on Distributed Resources for Bioinformatics Applications, *IEEE eScience 2008*, p. 229, 222.

[26] I. Raicu, Y. Zhao, C. Dumitrescu, I. Foster, and M. Wilde, 2007, Falkon: a Fast and Light-weight tasK executiON framework, In: *SC07*, p. 1-12, Reno, Nevada.

[27] D. Abramson, C. Enticott, and I. Altinas, 2008, Nimrod/K: towards massively parallel dynamic grid workflows, In: *SC 08*, p. 1-11, Austin, Texas.